

Grafos - algoritmos

BFS

```
from collections import deque

# número de vértices e arestas
n, m = map(int, input().split())

grafo = [[] for _ in range(n)]

# Leitura das arestas
for _ in range(m):
    u, v = map(int, input().split())
    grafo[u].append(v)
    grafo[v].append(u)

inicio = int(input())

visitado = [False] * n

fila = deque()
fila.append(inicio)
visitado[inicio] = True

while fila:
    v = fila.popleft()
    print(v, end=" ")

    for vizinho in grafo[v]:
        if not visitado[vizinho]:
            visitado[vizinho] = True
            fila.append(vizinho)
```

DFS

```
n, m = map(int, input().split())

grafo = [[] for _ in range(n)]

for _ in range(m):
    u, v = map(int, input().split())
    grafo[u].append(v)
    grafo[v].append(u)

inicio = int(input())

visitado = [False] * n

def dfs(v):
    visitado[v] = True
    print(v, end=" ")

    for vizinho in grafo[v]:
        if not visitado[vizinho]:
            dfs(vizinho)

dfs(inicio)
```

Ordenacao topologica

```
from collections import deque
```

```
q = deque()
```

```
for i in range(n):  
    if indegree[i] == 0:  
        q.append(i)
```

```
while q:  
    v = q.popleft()  
    for u in g[v]:  
        indegree[u] -= 1  
        if indegree[u] == 0:  
            q.append(u)
```

You are given a map of a building, and your task is to count the number of its rooms. The size of the map is $n \times m$ squares, and each square is either floor or wall. You can walk left, right, up, and down through the floor squares.

Input

The first input line has two integers n and m : the height and width of the map.

Then there are n lines of m characters describing the map. Each character is either `.` (floor) or `#` (wall).

Output

Print one integer: the number of rooms.

Constraints

$1 \leq n, m \leq 1000$

Example

Input:

5 8

```
#####  
#.#...#  
####.#.  
#.#...#  
#####
```

Output:

3

```
import sys  
from collections import deque  
  
input = sys.stdin.readline  
  
n, m = map(int, input().split())  
grid = [list(input().strip()) for _ in range(n)]  
visitado = [[False]*m for _ in range(n)]  
direcoes = [(1,0),(-1,0),(0,1),(0,-1)]  
  
salas = 0  
  
for i in range(n):  
    for j in range(m):  
        if grid[i][j] == '.' and not visitado[i][j]:
```

```

salas += 1
fila = deque([(i,j)])
visitado[i][j] = True

while fila:
    x, y = fila.popleft()

    for dx, dy in direcoes:
        nx = x + dx
        ny = y + dy

        if 0 <= nx < n and 0 <= ny < m:
            if grid[nx][ny] == '.' and not visitado[nx][ny]:
                visitado[nx][ny] = True
                fila.append((nx,ny))

print(salas)

```

You are given a map of a labyrinth, and your task is to find a path from start to end. You can walk left, right, up and down.

Input

The first input line has two integers n and m : the height and width of the map.

Then there are n lines of m characters describing the labyrinth. Each character is `.` (floor), `#` (wall), `A` (start), or `B` (end). There is exactly one `A` and one `B` in the input.

Output

First print "YES", if there is a path, and "NO" otherwise.

If there is a path, print the length of the shortest such path and its description as a string consisting of characters `L` (left), `R` (right), `U` (up), and `D` (down). You can print any valid solution.

Constraints

- $1 \leq n, m \leq 1000$

Example

Input	Output
<pre> 5 8 ##### #.A#...# #.#.#B# #.....# ##### </pre>	<pre> YES 9 LDDRRRRRU </pre>

```

import sys
from collections import deque
input = sys.stdin.readline
n, m = map(int, input().split())
A = [list(input().strip()) for _ in range(n)]
vis = [[False]*m for _ in range(n)]
previous_step = [[0]*m for _ in range(n)]

dx = [-1, 0, 1, 0]
dy = [0, 1, 0, -1]
step_dir = "URDL"

for i in range(n):
    for j in range(m):
        if A[i][j] == 'A':
            begin = (i, j)
        elif A[i][j] == 'B':
            end = (i, j)

fila = deque()
fila.append(begin)
vis[begin[0]][begin[1]] = True

while fila:
    x, y = fila.popleft()

    for i in range(4):
        nx = x + dx[i]
        ny = y + dy[i]

        if nx < 0 or nx >= n or ny < 0 or ny >= m:
            continue
        if A[nx][ny] == '#':
            continue
        if vis[nx][ny]:
            continue

        vis[nx][ny] = True
        previous_step[nx][ny] = i
        fila.append((nx, ny))

if vis[end[0]][end[1]]:
    print("YES")

```

```
steps = []

while end != begin:
    p = previous_step[end[0]][end[1]]
    steps.append(p)
    end = (end[0] - dx[p], end[1] - dy[p])
steps.reverse()
print(len(steps))

for c in steps:
    print(step_dir[c], end="")
print()

else:
    print("NO")
```

You and some monsters are in a labyrinth. When taking a step to some direction in the labyrinth, each monster may simultaneously take one as well. Your goal is to reach one of the boundary squares without ever sharing a square with a monster.

Your task is to find out if your goal is possible, and if it is, print a path that you can follow. Your plan has to work in any situation; even if the monsters know your path beforehand.

Input

The first input line has two integers n and m : the height and width of the map.

After this there are n lines of m characters describing the map. Each character is `.` (floor), `#` (wall), `A` (start), or `M` (monster). There is exactly one `A` in the input.

Output

First print "YES" if your goal is possible, and "NO" otherwise.

If your goal is possible, also print an example of a valid path (the length of the path and its description using characters `D`, `U`, `L`, and `R`). You can print any path, as long as its length is at most $n \cdot m$ steps.

Constraints

- $1 \leq n, m \leq 1000$

Example

Input	Output
5 8 ##### #M...A..# #..#.M..# ##### #..#####	YES S RRDDR

```

import sys
from collections import deque

input = sys.stdin.readline

n, m = map(int, input().split())
grid = [list(input().strip()) for _ in range(n)]

dx = [-1, 1, 0, 0]
dy = [0, 0, -1, 1]
dir_char = ['U', 'D', 'L', 'R']

monster_time = [[float('inf')] * m for _ in range(n)]

fila = deque()

for i in range(n):
    for j in range(m):
        if grid[i][j] == 'M':
            fila.append((i, j))
            monster_time[i][j] = 0
        if grid[i][j] == 'A':
            start = (i, j)

while fila:
    x, y = fila.popleft()

    for k in range(4):
        nx = x + dx[k]
        ny = y + dy[k]

        if 0 <= nx < n and 0 <= ny < m:
            if grid[nx][ny] != '#' and monster_time[nx][ny] == float('inf'):
                monster_time[nx][ny] = monster_time[x][y] + 1
                fila.append((nx, ny))

# BFS do jogador
fila = deque([start])
player_time = [[-1] * m for _ in range(n)]
parent = [[None] * m for _ in range(n)]
player_time[start[0]][start[1]] = 0

while fila:
    x, y = fila.popleft()
    if x == 0 or x == n - 1 or y == 0 or y == m - 1:
        end = (x, y)
        break

```

```

for k in range(4):
    nx = x + dx[k]
    ny = y + dy[k]
    if 0 <= nx < n and 0 <= ny < m:
        if grid[nx][ny] != '#' and player_time[nx][ny] == -1:

            if player_time[x][y] + 1 < monster_time[nx][ny]:
                player_time[nx][ny] = player_time[x][y] + 1
                parent[nx][ny] = (x,y,k)
                fila.append((nx,ny))
    else:
        print("NO")
        exit()
path = []
x,y = end

while (x,y) != start:
    px,py,k = parent[x][y]
    path.append(dir_char[k])
    x,y = px,py

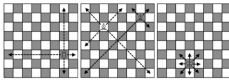
path.reverse()

print("YES")
print(len(path))
print("".join(path))

```

Little Petya is learning to play chess. He has already learned how to move a king, a rook and a bishop. Let us remind you the rules of moving chess pieces. A chessboard is 64 square fields organized into an 8 × 8 table. A field is represented by a pair of integers (r, c) — the number of the row and the number of the column (in a classical game the columns are traditionally indexed by letters). Each chess piece takes up exactly one field. To make a move is to move a chess piece, the pieces move by the following rules:

- A rook moves any number of fields horizontally or vertically.
- A bishop moves any number of fields diagonally.
- A king moves one field in any direction — horizontally, vertically or diagonally.



The pieces move like that.

Petya is thinking about the following problem: what minimum number of moves is needed for each of these pieces to move from field (r_1, c_1) to field (r_2, c_2) ? At that, we assume that there are no more pieces besides this one on the board. Help him solve this problem.

Input

The input contains four integers r_1, c_1, r_2, c_2 ($1 \leq r_1, c_1, r_2, c_2 \leq 8$) — the coordinates of the starting and the final field. The starting field doesn't coincide with the final one.

You can assume that the chessboard rows are numbered from top to bottom 1 through 8, and the columns are numbered from left to right 1 through 8.

Output

Print three space-separated integers: the minimum number of moves the rook, the bishop and the king (in this order) is needed to move from field (r_1, c_1) to field (r_2, c_2) . If a piece cannot make such a move, print a 0 instead of the corresponding number.

Examples

Input	Copy
4 3 1 6	
Output	Copy
2 1 3	
Input	Copy
5 5 5 6	
Output	Copy
0 8 1	

```

r1, c1, r2, c2 = map(int, input().split())

if r1 == r2 or c1 == c2:
    rook = 1
else:
    rook = 2

if (r1 + c1) % 2 != (r2 + c2) % 2:
    bishop = 0
elif abs(r1 - r2) == abs(c1 - c2):
    bishop = 1
else:
    bishop = 2

king = max(abs(r1 - r2), abs(c1 - c2))

print(rook, bishop, king)

```

In the city in which Alice and Bob live, the first metro line is being built. This metro line contains n stations numbered from 1 to n . Bob lives near the station with number 1, while Alice lives near the station with number s . The metro line has two tracks. Trains on the first track go from the station 1 to the station n and trains on the second track go in reverse direction. Just after the train arrives to the end of its track, it goes to the depot immediately, so it is impossible to travel on it after that.

Some stations are not yet open at all and some are only partially open — for each station and for each track it is known whether the station is closed for that track or not. If a station is closed for some track, all trains going in this track's direction pass the station without stopping on it.

When the Bob got the information on opened and closed stations, he found that traveling by metro may be unexpectedly complicated. Help Bob determine whether he can travel to the Alice's home by metro or he should search for some other transport.

Input

The first line contains two integers n and s ($2 \leq s \leq n \leq 1000$) — the number of stations in the metro and the number of the station where Alice's home is located. Bob lives at station 1.

Next lines describe information about closed and open stations.

The second line contains n integers $a_1, a_2, \dots, a_n$ ($a_i = 0$ or $a_i = 1$). If $a_i = 1$, then the i -th station is open on the first track (that is, in the direction of increasing station numbers). Otherwise the station is closed on the first track.

The third line contains n integers $b_1, b_2, \dots, b_n$ ($b_i = 0$ or $b_i = 1$). If $b_i = 1$, then the i -th station is open on the second track (that is, in the direction of decreasing station numbers). Otherwise the station is closed on the second track.

Output

Print "YES" (quotes for clarity) if Bob will be able to commute to the Alice's home by metro and "NO" (quotes for clarity) otherwise.

You can print each letter in any case (upper or lower).

Examples

input	Copy
5 3 1 1 1 1 1 1 1 1 1 1	
output	Copy
YES	

```
n, s = map(int, input().split())
a = list(map(int, input().split()))
b = list(map(int, input().split()))

s -= 1

if a[0] == 0:
    print("NO")
elif a[s] == 1:
    print("YES")
elif b[s] == 0:
    print("NO")
else:
    possible = False
    for i in range(s+1, n):
        if a[i] == 1 and b[i] == 1:
            possible = True
            break

    print("YES" if possible else "NO")
```


B. Hierarchy

time limit per test: 2 seconds

memory limit per test: 64 megabytes

Nick's company employed n people. Now Nick needs to build a tree hierarchy of «supervisor-surbordinate» relations in the company (this is to say that each employee, except one, has exactly one supervisor). There are m applications written in the following form: «employee a_i is ready to become a supervisor of employee b_i at extra cost c_i ». The qualification q_j of each employee is known, and for each application the following is true: $q_{a_i} > q_{b_i}$.

Would you help Nick calculate the minimum cost of such a hierarchy, or find out that it is impossible to build it.

Input

The first input line contains integer n ($1 \leq n \leq 1000$) — amount of employees in the company. The following line contains n space-separated numbers q_j ($0 \leq q_j \leq 10^6$) — the employees' qualifications. The following line contains number m ($0 \leq m \leq 10000$) — amount of received applications. The following m lines contain the applications themselves, each of them in the form of three space-separated numbers: a_i , b_i and c_i ($1 \leq a_i, b_i \leq n$, $0 \leq c_i \leq 10^6$). Different applications can be similar, i.e. they can come from one and the same employee who offered to become a supervisor of the same person but at a different cost. For each application $q_{a_i} > q_{b_i}$.

Output

Output the only line — the minimum cost of building such a hierarchy, or -1 if it is impossible to build it.

Examples

input	Copy
<pre>4 7 2 3 1 4 1 2 5 2 4 1 3 4 1 1 3 5</pre>	
output	Copy
<pre>11</pre>	

```
import sys

input = sys.stdin.readline

n = int(input())
q = list(map(int, input().split()))

m = int(input())

INF = 10**18
min_cost = [INF]*n

for _ in range(m):
    a,b,c = map(int,input().split())
    b -= 1
    min_cost[b] = min(min_cost[b], c)

root = q.index(max(q))

ans = 0
```

```

for i in range(n):
    if i == root:
        continue
    if min_cost[i] == INF:
        print(-1)
        exit()
    ans += min_cost[i]

print(ans)

```

Sua tarefa é encontrar a rota de voo mais barata de Syrjälä para Metsälä. Você tem um cupom de desconto, com o qual pode reduzir pela metade o preço de qualquer voo durante a rota. No entanto, você só pode usar o cupom uma vez.

Quando você usa o cupom de desconto para um voo cujo preço é x , seu preço se torna $\lfloor x/2 \rfloor$ (arredondado para baixo para um número inteiro).

Entrada

A primeira linha de entrada contém dois inteiros n e m : o número de cidades e conexões de voo. As cidades são numeradas de $1, 2, \dots, n$. A cidade 1 é Syrjälä, e a cidade n é Metsälä.

Depois disso, há m linhas descrevendo os voos. Cada linha contém três inteiros a, b e c : um voo começa na cidade a , termina na cidade b e seu preço é c . Cada voo é unidirecional.

É possível sempre chegar de Syrjälä a Metsälä.

Saída

Imprima um inteiro: o preço da rota mais barata de Syrjälä para Metsälä.

Restrições

- $2 \leq n \leq 10^5$
- $1 \leq m \leq 2 \cdot 10^5$
- $1 \leq a, b \leq n$
- $1 \leq c \leq 10^9$

Exemplo

Input	Output
<pre> 3 4 1 2 3 2 3 1 1 3 7 2 1 5 </pre>	<pre> 2 </pre>

```

import sys
import heapq

input = sys.stdin.readline

n, m = map(int, input().split())

g = [[] for _ in range(n)]
gr = [[] for _ in range(n)]
edges = []

for _ in range(m):
    a, b, c = map(int, input().split())
    a -= 1
    b -= 1
    g[a].append((b, c))
    gr[b].append((a, c))
    edges.append((a, b, c))

```

```

INF = 10**18

def dijkstra(graph, start):
    dist = [INF]*n
    dist[start] = 0
    pq = [(0, start)]

    while pq:
        d,u = heapq.heappop(pq)
        if d > dist[u]:
            continue

        for v,w in graph[u]:
            nd = d + w
            if nd < dist[v]:
                dist[v] = nd
                heapq.heappush(pq,(nd,v))

    return dist

dist1 = dijkstra(g, 0)
dist2 = dijkstra(gr, n-1)

ans = INF

for a,b,c in edges:
    ans = min(ans, dist1[a] + c//2 + dist2[b])

print(ans)

```

Há n cidades e m estradas entre elas. Infelizmente, a condição das estradas é tão ruim que elas não podem ser usadas. Sua tarefa é consertar algumas das estradas para que haja uma rota decente entre quaisquer duas cidades.

Para cada estrada, você conhece o custo de reparação, e você deve encontrar uma solução onde o custo total seja o menor possível.

Entrada

A primeira linha de entrada tem dois inteiros n e m : o número de cidades e estradas. As cidades são numeradas de $1, 2, \dots, n$.

Em seguida, há m linhas descrevendo as estradas. Cada linha tem três inteiros a, b e c : há uma estrada entre as cidades a e b , e seu custo de reparação é c . Todas as estradas são de mão dupla.

Cada estrada está entre duas cidades diferentes, e há no máximo uma estrada entre duas cidades.

Saída

Imprima um inteiro: o custo total mínimo de reparação. No entanto, se não houver soluções, imprima "IMPOSSIVEL".

Restrições

- $1 \leq n \leq 10^5$
- $1 \leq m \leq 2 \cdot 10^5$
- $1 \leq a, b \leq n$
- $1 \leq c \leq 10^9$

Exemplo

Input	copy	Output	copy
<pre> 5 6 1 2 3 2 3 5 2 4 2 3 4 8 5 1 7 5 4 4 </pre>		14	

```

import sys
input = sys.stdin.readline
n, m = map(int, input().split())
edges = []
for _ in range(m):
    a,b,c = map(int,input().split())
    edges.append((c,a-1,b-1))

edges.sort()
parent = list(range(n))
rank = [0]*n

```

```

def find(x):
    while parent[x] != x:
        parent[x] = parent[parent[x]]
        x = parent[x]
    return x
def union(a,b):
    ra, rb = find(a), find(b)
    if ra == rb:
        return False
    if rank[ra] < rank[rb]:
        parent[ra] = rb
    else:
        parent[rb] = ra
        if rank[ra] == rank[rb]:
            rank[ra] += 1
    return True

cost = 0
edges_used = 0

for c,a,b in edges:
    if union(a,b):
        cost += c
        edges_used += 1

if edges_used == n-1:
    print(cost)
else:
    print("IMPOSSIBLE")

```

Há n cidades e m conexões de voo entre elas. Sua tarefa é determinar o comprimento da rota mais curta de Syrjälä para cada cidade.

Entrada

A primeira linha de entrada contém dois inteiros n e m : o número de cidades e conexões de voo. As cidades são numeradas de $1, 2, \dots, n$, e a cidade 1 é Syrjälä.

Depois, há m linhas descrevendo as conexões de voo. Cada linha contém três inteiros a, b e c : um voo começa na cidade a , termina na cidade b e tem comprimento c . Cada voo é de sentido único.

Você pode assumir que é possível viajar de Syrjälä para todas as outras cidades.

Saída

Imprima n inteiros: os comprimentos das rotas mais curtas de Syrjälä para as cidades $1, 2, \dots, n$.

Restrições

- $1 \leq n \leq 10^5$
- $1 \leq m \leq 2 \cdot 10^5$
- $1 \leq a, b \leq n$
- $1 \leq c \leq 10^9$

Exemplo

Input	copy	Output	copy
3 4 1 2 6 1 3 2 3 2 3 1 3 4		0 5 2	

```

import sys
import heapq

input = sys.stdin.readline

n, m = map(int, input().split())

g = [[] for _ in range(n)]

for _ in range(m):
    a,b,c = map(int,input().split())
    g[a-1].append((b-1,c))

INF = 10**18
dist = [INF]*n
dist[0] = 0

```



```

        cells_to_open -= 1

        if cells_to_open == 0:
            break

# converter para string novamente
for i in range(n):
    maze[i] = ''.join(maze[i])

print('\n'.join(maze))

```

Há n cidades e m conexões de voo. Sua tarefa é verificar se é possível viajar de qualquer cidade para qualquer outra cidade usando os voos disponíveis.

Entrada

A primeira linha de entrada contém dois inteiros n e m : o número de cidades e voos. As cidades são numeradas de $1, 2, \dots, n$.

Após isso, há m linhas descrevendo os voos. Cada linha contém dois inteiros a e b : há um voo da cidade a para a cidade b . Todos os voos são voos de ida.

Saída

Imprima "SIM" se todas as rotas forem possíveis, e "NÃO" caso contrário. Neste último caso, também imprima duas cidades a e b tais que você não pode viajar da cidade a para a cidade b . Se houver várias soluções possíveis, você pode imprimir qualquer uma delas.

Restrições

- $1 \leq n \leq 10^5$
- $1 \leq m \leq 2 \cdot 10^5$
- $1 \leq a, b \leq n$

Exemplo

Input	copy	Output	copy
<pre> 4 5 1 2 2 3 3 1 1 4 3 4 </pre>		<pre> NO 4 2 </pre>	

```

import sys
sys.setrecursionlimit(10**7)

n, m = map(int, input().split())

graph = [[] for _ in range(n+1)]
reverse_graph = [[] for _ in range(n+1)]

for _ in range(m):
    a, b = map(int, input().split())
    graph[a].append(b)
    reverse_graph[b].append(a)

visited = [False]*(n+1)

def dfs(v, g):
    visited[v] = True
    for u in g[v]:
        if not visited[u]:
            dfs(u, g)

dfs(1, graph)

for i in range(1, n+1):
    if not visited[i]:
        print("NO")
        print(1, i)
        exit()

visited = [False]*(n+1)

dfs(1, reverse_graph)

```

```

for i in range(1, n+1):
    if not visited[i]:
        print("NO")
        print(i, 1)
        exit()

```

```

print("YES")

```

Há n cidades e m estradas entre elas. Sua tarefa é processar q consultas onde você deve determinar o comprimento da rota mais curta entre duas cidades dadas.

Entrada

A primeira linha de entrada contém três inteiros n , m e q : o número de cidades, estradas e consultas.

Em seguida, há m linhas descrevendo as estradas. Cada linha contém três inteiros a , b e c : há uma estrada entre as cidades a e b cujo comprimento é c . Todas as estradas são de mão dupla.

Por fim, há q linhas descrevendo as consultas. Cada linha contém dois inteiros a e b : determine o comprimento da rota mais curta entre as cidades a e b .

Saída

Imprima o comprimento da rota mais curta para cada consulta. Se não houver rota, imprima -1 em seu lugar.

Restrições

- $1 \leq n \leq 500$
- $1 \leq m \leq n^2$
- $1 \leq q \leq 10^5$
- $1 \leq a, b \leq n$
- $1 \leq c \leq 10^9$

Exemplo

Input	Output
<pre> 4 3 5 1 2 5 1 3 9 2 3 3 1 2 2 1 1 3 1 4 3 2 </pre>	<pre> 5 5 8 -1 3 </pre>

```

import sys
input = sys.stdin.readline

n, m, q = map(int, input().split())

INF = 10**18
dist = [[INF]*(n+1) for _ in range(n+1)]

for i in range(1, n+1):
    dist[i][i] = 0

for _ in range(m):
    a, b, c = map(int, input().split())
    if c < dist[a][b]:
        dist[a][b] = c
        dist[b][a] = c

for k in range(1, n+1):
    dk = dist[k]
    for i in range(1, n+1):
        dik = dist[i][k]
        if dik == INF:
            continue
        di = dist[i]
        for j in range(1, n+1):
            new = dik + dk[j]
            if new < di[j]:
                di[j] = new

for _ in range(q):
    a, b = map(int, input().split())
    ans = dist[a][b]
    print(ans if ans != INF else -1)

```

Um jogo tem n planetas, conectados por m teleportadores. Dois planetas a e b pertencem ao mesmo reino exatamente quando há uma rota tanto de a para b quanto de b para a . Sua tarefa é determinar para cada planeta seu reino.

Entrada

A primeira linha de entrada contém dois inteiros n e m : o número de planetas e teleportadores. Os planetas são numerados de $1, 2, \dots, n$.

Após isso, há m linhas descrevendo os teleportadores. Cada linha contém dois inteiros a e b : você pode viajar do planeta a para o planeta b através de um teleportador.

Saída

Primeiro imprima um inteiro k : o número de reinos. Após isso, imprima para cada planeta um rótulo de reino entre 1 e k . Você pode imprimir qualquer solução válida.

Restrições

- $1 \leq n \leq 10^5$
- $1 \leq m \leq 2 \cdot 10^5$
- $1 \leq a, b \leq n$

Exemplo

Input	copy	Output	copy
5 6 1 2 2 3 3 1 3 4 4 5 5 4		2 1 1 1 2 2	

```
import sys
sys.setrecursionlimit(10**7)
input = sys.stdin.readline

n, m = map(int, input().split())

graph = [[] for _ in range(n+1)]
rev_graph = [[] for _ in range(n+1)]

for _ in range(m):
    a, b = map(int, input().split())
    graph[a].append(b)
    rev_graph[b].append(a)

visited = [False]*(n+1)
order = []

def dfs1(v):
    visited[v] = True
    for u in graph[v]:
        if not visited[u]:
            dfs1(u)
    order.append(v)

for i in range(1, n+1):
    if not visited[i]:
        dfs1(i)

comp = [0]*(n+1)
visited = [False]*(n+1)
k = 0

def dfs2(v):
    comp[v] = k
    visited[v] = True
    for u in rev_graph[v]:
        if not visited[u]:
            dfs2(u)

for v in reversed(order):
    if not visited[v]:
        k += 1
        dfs2(v)

print(k)
print(*comp[1:])
```


H - Valera and Elections [CodeForces - 369C](#)

The city Valera lives in is going to hold elections to the city Parliament.

The city has n districts and $n - 1$ bidirectional roads. We know that from any district there is a path along the roads to any other district. Let's enumerate all districts in some way by integers from 1 to n , inclusive. Furthermore, for each road the residents decided if it is the problem road or not. A *problem road* is a road that needs to be repaired.

There are n candidates running the elections. Let's enumerate all candidates in some way by integers from 1 to n , inclusive. If the candidate number i will be elected in the city Parliament, he will perform exactly one promise — to repair all problem roads on the way from the i -th district to the district 1, where the city Parliament is located.

Help Valera and determine the subset of candidates such that if all candidates from the subset will be elected to the city Parliament, all problem roads in the city will be repaired. If there are several such subsets, you should choose the subset consisting of the minimum number of candidates.

Input

The first line contains a single integer n ($2 \leq n \leq 10^5$) — the number of districts in the city.

Then $n - 1$ lines follow. Each line contains the description of a city road as three positive integers x_i, y_i, t_i ($1 \leq x_i, y_i \leq n, 1 \leq t_i \leq 2$) — the districts connected by the i -th bidirectional road and the road type. If t_i equals to one, then the i -th road isn't the problem road; if t_i equals to two, then the i -th road is the problem road.

It's guaranteed that the graph structure of the city is a tree.

Output

In the first line print a single non-negative number k — the minimum size of the required subset of candidates. Then on the second line print k space-separated integers $a_1, a_2, \dots, a_k$ — the numbers of the candidates that form the required subset. If there are multiple solutions, you are allowed to print any of them.

```
import sys
sys.setrecursionlimit(200000)

n = int(sys.stdin.readline())

g = [[] for _ in range(n + 1)]

for _ in range(n - 1):
    x, y, t = map(int, sys.stdin.readline().split())
    g[x].append((y, t))
    g[y].append((x, t))

ans = []

def dfs(v, parent):
    has_problem = False

    for to, t in g[v]:
```

```

        if to == parent:
            continue

        child_has = dfs(to, v)

        if t == 2 and not child_has:
            ans.append(to)

        if t == 2 or child_has:
            has_problem = True

    return has_problem

dfs(1, -1)

print(len(ans))
print(*ans)

```

outra resposta:

```

import sys
sys.setrecursionlimit(100100)
broken=[0 for i in range(100001)]
adj=[[ ] for i in range(100001)]
def dfs(parent):
    visited.add(parent)
    for child in adj[parent]:
        if child not in visited:
            broken[parent]+=dfs(child)
    return(broken[parent])
for _ in range(int(input())-1):
    a,b,c=map(int,input().split())
    adj[a].append(b)
    adj[b].append(a)
    if c==2:
        broken[b]=1
        broken[a]=1
visited=set()
dfs(1)
ans=[]
for i in range(100001):
    if broken[i]==1:
        ans.append(i)
print(len(ans))
print(*ans)

```

F - Connect Codeforces - 1130C

Alice lives on a flat planet that can be modeled as a square grid of size $n \times n$, with rows and columns enumerated from 1 to n . We represent the cell at the intersection of row r and column c with ordered pair (r, c) . Each cell in the grid is either *land* or *water*.

(1, 1)	(1, 2)	(1, 3)	(1, 4)	(1, 5)
(2, 1)	(2, 2)	(2, 3)	(2, 4)	(2, 5)
(3, 1)	(3, 2)	(3, 3)	(3, 4)	(3, 5)
(4, 1)	(4, 2)	(4, 3)	(4, 4)	(4, 5)
(5, 1)	(5, 2)	(5, 3)	(5, 4)	(5, 5)

An example planet with $n = 5$. It also appears in the first sample test.

Alice resides in *land* cell (r_1, c_1) . She wishes to travel to *land* cell (r_2, c_2) . At any moment, she may move to one of the cells adjacent to where she is—in one of the four directions (i.e., up, down, left, or right).

Unfortunately, Alice cannot swim, and there is no viable transportation means other than by foot (i.e., she can walk only on *land*). As a result, Alice's trip may be impossible.

To help Alice, you plan to create at **most one** tunnel between some two *land* cells. The tunnel will allow Alice to freely travel between the two endpoints. Indeed, creating a tunnel is a lot of effort: the cost of creating a tunnel between cells (r_s, c_s) and (r_t, c_t) is $(r_s - r_t)^2 + (c_s - c_t)^2$.

For now, your task is to find the minimum possible cost of creating at **most one** tunnel so that Alice could travel from (r_1, c_1) to (r_2, c_2) . If no tunnel needs to be created, the cost is 0.

Input

The first line contains one integer n ($1 \leq n \leq 50$) — the width of the square grid.

The second line contains two space-separated integers r_1 and c_1 ($1 \leq r_1, c_1 \leq n$) — denoting the cell where Alice resides.

The third line contains two space-separated integers r_2 and c_2 ($1 \leq r_2, c_2 \leq n$) — denoting the cell to which Alice wishes to travel.

```
import sys
sys.setrecursionlimit(10**7)

def dfs(x, y, visited, grid, n):
    if x < 1 or x > n or y < 1 or y > n:
        return
    if grid[x-1][y-1] == '1' or (x, y) in visited:
        return

    visited.add((x, y))

    for dx, dy in [(-1,0),(1,0),(0,-1),(0,1)]:
        dfs(x+dx, y+dy, visited, grid, n)

n = int(input())
r1, c1 = map(int, input().split())
r2, c2 = map(int, input().split())

grid = [input().strip() for _ in range(n)]

comp1 = set()
comp2 = set()

dfs(r1, c1, comp1, grid, n)
dfs(r2, c2, comp2, grid, n)

ans = float('inf')

for x1, y1 in comp1:
    for x2, y2 in comp2:
        ans = min(ans, (x1-x2)**2 + (y1-y2)**2)

print(ans)
```